

프론트엔드 입문

React 기초 개념

실습을 시작하기 전에 알아야 할 핵심 개념 정리



컴포넌트 · JSX · props · state · 훅 · 이벤트 · 리스트



React를 쓰는 이유

지난 시간에 손으로 DOM을 조작하며 To-Do를 만들었습니다. React는 그 반복 작업을 줄여 줍니다.

화면을 묘사만

어떻게 그릴지 대신, 무엇을 보여 줄지만 적습니다.

자동으로 갱신

데이터가 바뀌면 화면을 React가 다시 그립니다.

조각으로 재사용

화면을 작은 컴포넌트로 나눠 다시 씁니다.

핵심 한 줄: 화면은 데이터의 결과입니다. 데이터를 바꾸면 화면이 따라옵니다.

React 기초, 다섯 가지

이 다섯 가지를 차례로 보겠습니다. 모두 실습에서 바로 쓰는 개념입니다.

1 컴포넌트

화면을 만드는 조각

2 JSX

화면의 모습을 적는 문법

3 props

컴포넌트에 값을 전달

4 state · 훅

변하는 데이터를 다루기

5 이벤트 · 리스트

상호작용과 목록 그리기

컴포넌트: 화면을 만드는 조각

컴포넌트는 JSX를 return하는 함수입니다. 한 번 만들어 태그처럼 사용합니다.

```
function Hello() {  
  return <h1>안녕하세요</h1>;  
}
```

```
// 사용  
<Hello />
```

- ✓ 함수로 만듭니다
- ✓ 이름은 대문자로 시작합니다
- ✓ JSX를 return합니다
- ✓ <Hello /> 처럼 태그로 사용합니다
- ✓ 여러 번 재사용할 수 있습니다

</> JSX: 화면의 모습을 적는 문법

HTML처럼 보이지만 JavaScript입니다. 중괄호로 값을 끼워 넣습니다.

```
const name = "수진";  
const el = <h1>{name}님 환영합니다</h1>;
```

자주 틀리는 규칙 네 가지

1. 하나로 감싸기 `<> ... </>`

2. class 대신 className `className="box"`

3. 값은 중괄호로 `{name}`

4. 태그는 닫기 `<img />`

↓ props: 컴포넌트에 값 전달

부모가 자식 컴포넌트에 값을 내려 줍니다. 자식은 props로 받아 사용합니다.

```
function Welcome(props) {  
  return <h1>{props.name}님 환영합니다</h1>;  
}
```

```
// 사용: name 이라는 값을 내려 줍니다  
<Welcome name="수진" />
```

- ✓ 부모 → 자식 방향으로 전달합니다
- ✓ 태그 속성처럼 적습니다
- ✓ 자식은 props로 받습니다
- ✓ props는 읽기 전용입니다
- ✓ 같은 컴포넌트를 값만 바꿔 재사용합니다



state: 변하는 데이터

props가 밖에서 받은 값이라면, state는 컴포넌트가 스스로 가진 변하는 값입니다.

props

밖에서 받은 값. 자식이 바꿀 수 없습니다.

state

스스로 가진 값. 바뀌면 화면이 다시 그려집니다.

핵심: state를 바꾸면 화면은 자동으로 다시 그려집니다. 손으로 새로 그릴 필요가 없습니다.

훅과 useState

훅은 함수 컴포넌트에 기능을 더해 주는 함수입니다. 이름이 use로 시작합니다. 가장 많이 쓰는 훅이 state를 다루는 useState입니다.

```
import { useState } from "react";  
  
const [count, setCount] = useState(0);  
//      값           바꾸는 함수       처음 값
```

- ✓ 이름이 use로 시작합니다
- ✓ useState는 [값, 바꾸는 함수]를 줍니다
- ✓ setCount로 바꾸면 화면이 갱신됩니다
- ✓ useEffect 등 다른 훅도 있습니다

이벤트 처리

클릭 같은 동작은 JSX 안에서 onClick으로 연결합니다. 카멜케이스로 적고 함수를 넘깁니다.

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      {count}번 클릭  
    </button>  
  );  
}
```

- ✓ JSX 안에서 onClick으로 연결합니다
- ✓ onClick처럼 카멜케이스로 적습니다
- ✓ 실행할 함수를 넘깁니다
- ✓ 버튼을 누르면 state가 바뀝니다
- ✓ state가 바뀌니 화면도 갱신됩니다

☰ 리스트 렌더링

배열을 map으로 JSX 목록으로 바꿉니다. 각 항목에는 key가 필요합니다.

```
<ul>
  {todos.map((todo) => (
    <li key={todo.id}>{todo.text}</li>
  ))}
</ul>
```

- ✓ 배열을 map으로 변환합니다
- ✓ 결과는 JSX 요소의 배열입니다
- ✓ 각 항목에 key를 붙입니다
- ✓ 우리 To-Do의 forEach가 이걸로 바뀝니다

전체 그림: 화면은 데이터의 결과

지금까지 본 개념이 하나로 이어집니다. state가 바뀌면 React가 화면을 다시 그립니다.



그래서 바닐라에서 부르던 *refresh* 같은 다시 그리기 작업이 사라집니다.



실습으로

이제 To-Do를 React로 직접 만들어 봅니다. 오늘 개념을 그대로 사용합니다.

- ✓ 컴포넌트로 화면을 나눕니다
- ✓ JSX로 모습을 적습니다
- ✓ useState로 할 일 목록을 state로 둡니다
- ✓ onClick으로 추가·삭제·완료를 처리합니다
- ✓ map과 key로 목록을 그립니다

준비되었으면 실습 파일을 열어 시작하겠습니다.

